САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №2

Выполнила:

Автономова Ксения

J3213

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В лабораторной работе №2 мне нужно было доработать сайт, который я сделала в первой лабораторной работе, и привязать его к внешнему API. Если в первой лабораторной работе основной задачей была вёрстка интерфейса средствами HTML, CSS, Bootstrap и базового JavaScript, то во второй лабораторной работе нужно было реализовать получение и отправку данных через API, а также подключить авторизацию. Смысл работы состоял в том, чтобы взять уже готовый интерфейс из ЛР1 и сделать его более реалистичным за счёт работы с данными.

По заданию нужно было:

- использовать формат данных JSON;
- реализовать взаимодействие с внешним API средствами fetch, axios или xhr;
- сделать моковое API при помощи json-server;
- подключить авторизацию, как в примерах по теме имитации работы с закрытым API.

Таким образом, если в первой лабораторной работе у меня был в основном интерфейс сайта, то во второй лабораторной работе этот интерфейс был превращён в более полноценное приложение, в котором данные о пользователях, мероприятиях, билетах, возвратах и отзывах уже не просто показываются на странице, а загружаются и изменяются через mock API.

Итоговая задача для меня состояла в том, чтобы сохранить интерфейс, разработанный в ЛР1, но при этом заменить демонстрационную логику на реальную работу с сервером, реализованным через JSON Server.

Ход работы

1. **Общая идея работы и связь с лабораторной работой №1**

При выполнении второй лабораторной я продолжила работать с темой платформы для покупки билетов на мероприятия в Санкт-Петербурге.

В первой лабораторной работе мной были сделаны:

- главная страница;
- вход;
- регистрация;
- личный кабинет пользователя;
- поиск мероприятий;
- страница отдельного мероприятия;
- кабинет организатора.

Однако тогда эти страницы выполняли в основном демонстрационную функцию. Например:

- вход и регистрация были реализованы как форма с базовой проверкой;
- поиск работал на локальных данных;
- билеты и возвраты были показаны как интерфейс;
- действия пользователя выполнялись без реального API.

Во второй лабораторной работе я сохранила внешний вид и общую структуру сайта, но изменила его внутреннюю логику. Теперь:

- данные мероприятий загружаются с сервера;
- регистрация и вход работают через API;
- покупка билета записывается в базу;
- возврат билета тоже отправляется на сервер;

- отзывы к мероприятиям подгружаются и сохраняются через API;
- организатор может создавать мероприятия, и они появляются в системе.

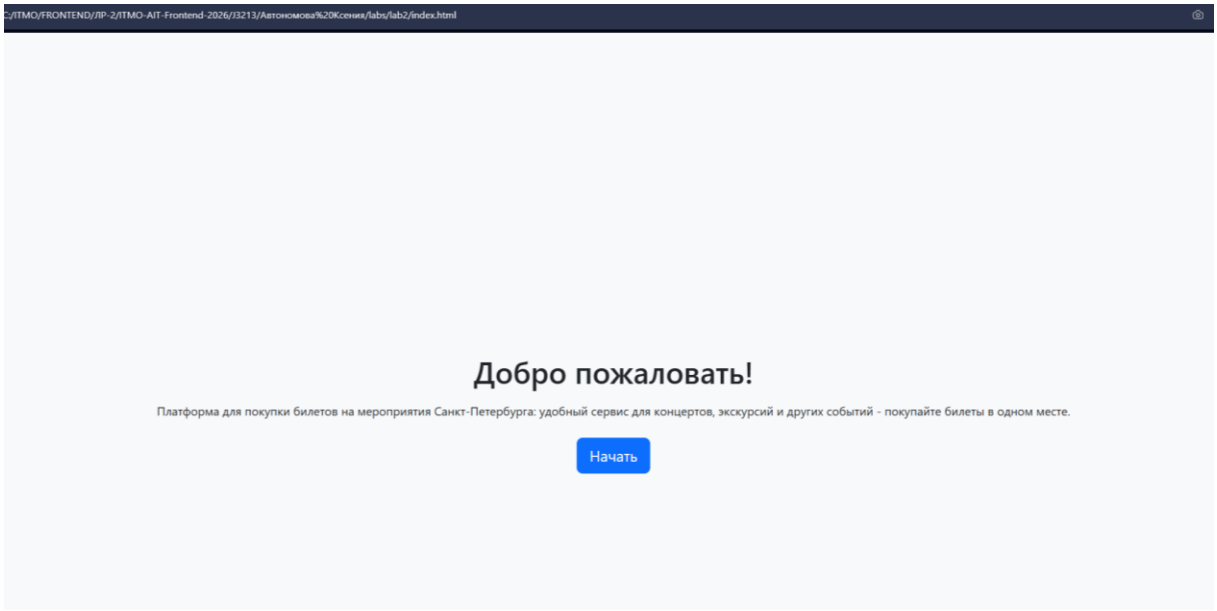


Рисунок 1 - Главная страница сайта.

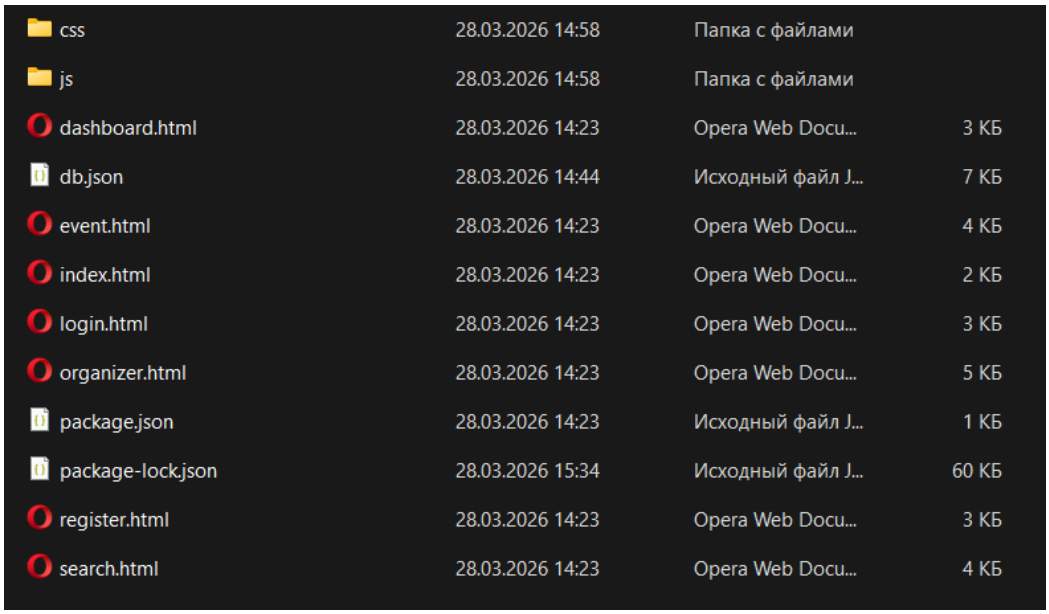
2. Построение структуры проекта

Так как во второй лабораторной работе проект стал более сложным, я решила сделать его структуру более удобной и логичной. По сравнению с первой лабораторной работой структура была расширена.

В итоговом проекте были использованы следующие основные файлы:

- index.html - главная страница;
- login.html - страница входа;
- register.html - страница регистрации;
- dashboard.html - личный кабинет пользователя;
- search.html - страница поиска мероприятий;

- event.html - страница отдельного мероприятия;
- organizer.html - кабинет организатора;
- css/style.css - файл стилей;
- js/script.js - общий JavaScript для интерфейса;
- js/auth-service.js - работа с авторизацией и регистрацией через API;
- js/auth-ui.js - обработка форм входа и регистрации;
- js/api.js - основные функции взаимодействия с API;
- js/search-page.js - логика страницы поиска;
- js/event-page.js - логика страницы мероприятия;
- js/dashboard-page.js - логика личного кабинета пользователя;
- js/organizer-page.js - логика кабинета организатора;
- db.json - mock база данных для JSON Server;
- package.json - конфигурация проекта и запуск сервера;
- package-lock.json - файл зависимостей.



css	28.03.2026 14:58	Папка с файлами	
js	28.03.2026 14:58	Папка с файлами	
dashboard.html	28.03.2026 14:23	Opera Web Docu...	3 КБ
db.json	28.03.2026 14:44	Исходный файл J...	7 КБ
event.html	28.03.2026 14:23	Opera Web Docu...	4 КБ
index.html	28.03.2026 14:23	Opera Web Docu...	2 КБ
login.html	28.03.2026 14:23	Opera Web Docu...	3 КБ
organizer.html	28.03.2026 14:23	Opera Web Docu...	5 КБ
package.json	28.03.2026 14:23	Исходный файл J...	1 КБ
package-lock.json	28.03.2026 15:34	Исходный файл J...	60 КБ
register.html	28.03.2026 14:23	Opera Web Docu...	3 КБ
search.html	28.03.2026 14:23	Opera Web Docu...	4 КБ

Рисунок 2 - Структура проекта лабораторной работы №2.

3. Реализация mock API средствами JSON Server

Одним из главных отличий ЛР2 от ЛР1 стало подключение mock API. Для этого я использовала json-server и json-server-auth.

В файле package.json я настроила запуск сервера через команду:

```
json-server db.json -m ./node_modules/json-server-auth
```

Это позволило использовать:

- обычные открытые маршруты для чтения общедоступных данных;
- защищённые маршруты для действий, которые доступны только после входа в систему.

Основным файлом данных стал db.json. В нём были созданы коллекции:

- users - пользователи;
- events - мероприятия;
- tickets - купленные билеты;
- returns - возвращённые билеты;
- reviews - отзывы.

Именно эта база данных теперь используется страницами сайта вместо условных или локальных данных, как это было в ЛР1.

Таким образом, во второй лабораторной работе сайт начал работать уже не как просто свёрстанный интерфейс, а как приложение, которое действительно получает и изменяет данные.

```
  Type s + enter at any time to create a snapshot of the database
POST /login 200 73.029 ms - 362
POST /login 200 49.996 ms - 362
GET /events 304 26.652 ms - -
GET /600/tickets 304 27.998 ms - -
GET /600/returns 304 29.312 ms - -
```

Рисунок 3 - Терминал с запущенным JSON Server (хорошо показывает динамическое выполнение, успешные GET/POST/DELETE-запросы).

4. Реализация авторизации и регистрации

Если в первой лабораторной работе вход и регистрация были в основном демонстрационными, то во второй лабораторной работе я реализовала их через API.

Для этого были созданы отдельные файлы:

- auth-service.js - отвечает за отправку запросов на /signup и /login;
- auth-ui.js - отвечает за обработку форм на страницах входа и регистрации.

На странице входа пользователь вводит:

- адрес электронной почты;
- пароль;
- тип пользователя.

После этого выполняется запрос на сервер, и если данные корректны, сохраняются:

- токен доступа;

- данные пользователя;
- роль пользователя.

На странице регистрации пользователь вводит:

- ФИО;
- адрес электронной почты;
- пароль;
- подтверждение пароля;
- тип пользователя.

Также сохраняется роль: обычный пользователь или организатор.

По сравнению с ЛР1 это важное изменение, потому что раньше после нажатия на кнопку происходил только переход на нужную страницу, а теперь используется полноценный mock API с авторизацией.

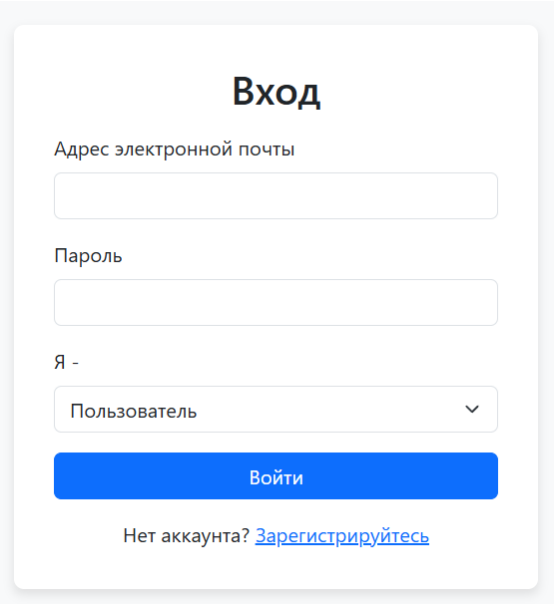


Рисунок 4 - Страница входа пользователя.

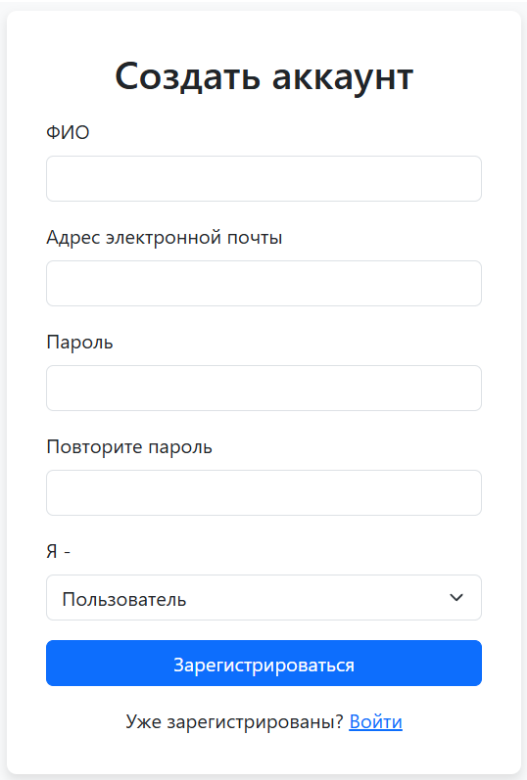


Рисунок 5 - Страница регистрации пользователя.

5. Страница поиска мероприятий

В первой лабораторной работе страница поиска уже была реализована визуально и содержала фильтрацию по:

- типу;
- дате;
- месту.

Во второй лабораторной работе я сохранила сам интерфейс поиска, но изменила источник данных. Теперь мероприятия загружаются через API, а фильтрация выполняется уже после получения списка событий с сервера.

Для логики этой страницы был выделен отдельный файл search-page.js.
На странице используются:

- форма фильтрации;
- карточки мероприятий;
- динамическое обновление списка при изменении фильтров.

Каждая карточка по-прежнему содержит:

- афишу;
- название;
- тип;
- дату;
- место проведения;
- краткое описание;
- кнопку перехода к подробной странице.

По сравнению с ЛР1 страница визуально осталась похожей, но теперь она уже работает не на условных встроенных данных, а на данных из API.

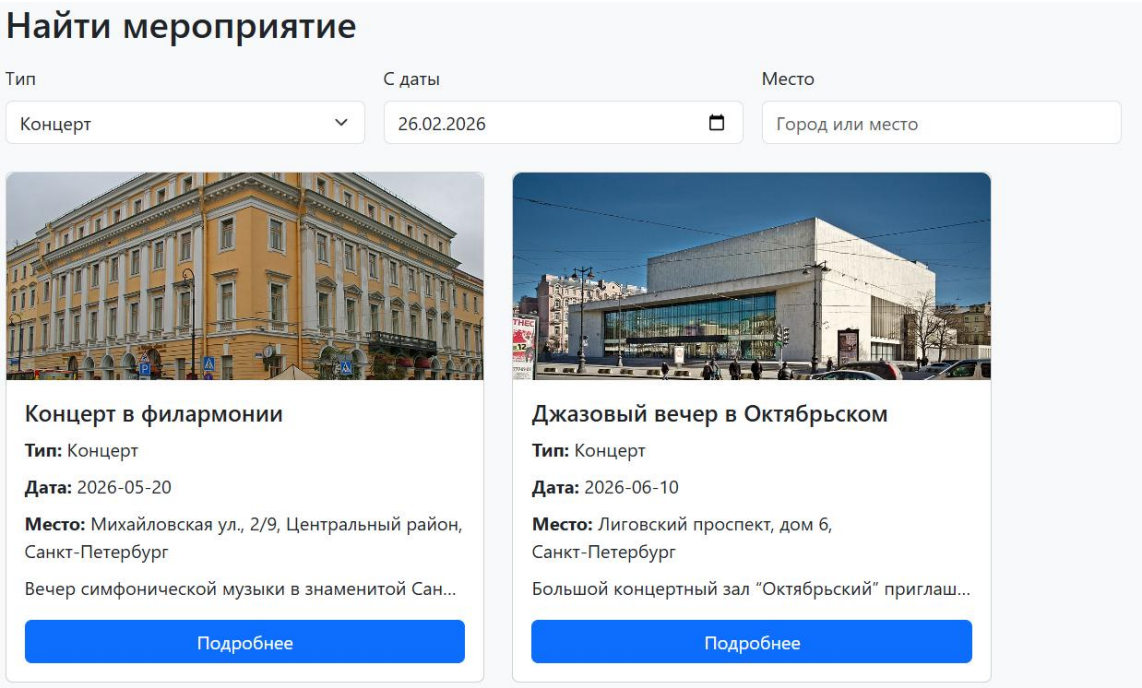


Рисунок 6 - Поиск мероприятий с фильтрацией и загрузкой данных через API.

6. Страница мероприятия

Одной из самых важных страниц проекта является страница отдельного мероприятия. В первой лабораторной работе она в основном показывала интерфейс и демонстрационные данные. Во второй лабораторной работе эта страница была доработана и стала работать через API.

Для неё был создан отдельный файл event-page.js.

На странице мероприятия теперь отображаются:

- афиша;
- название;
- тип;

- дата;
- место проведения;
- цена;
- описание;
- схема зала;
- отзывы.

Данные о самом событии загружаются с сервера по id мероприятия. Также через API загружаются отзывы и уже занятые места.

Если мероприятие относится к типу «концерт», отображается схема зала. При этом:

- пользователь может выбрать место;
- уже занятые места становятся недоступными;
- после покупки билет сохраняется на сервере;
- после возврата место снова может стать доступным.

Для мероприятий без мест покупка выполняется без выбора места.

Кроме того, в финальной логике учитывается разделение ролей. Организатор не должен покупать билеты, поэтому такая возможность ограничивается.

По сравнению с ЛР1 эта страница стала одной из самых функционально насыщенных, потому что теперь она не просто показывает макет, а действительно взаимодействует с сервером.

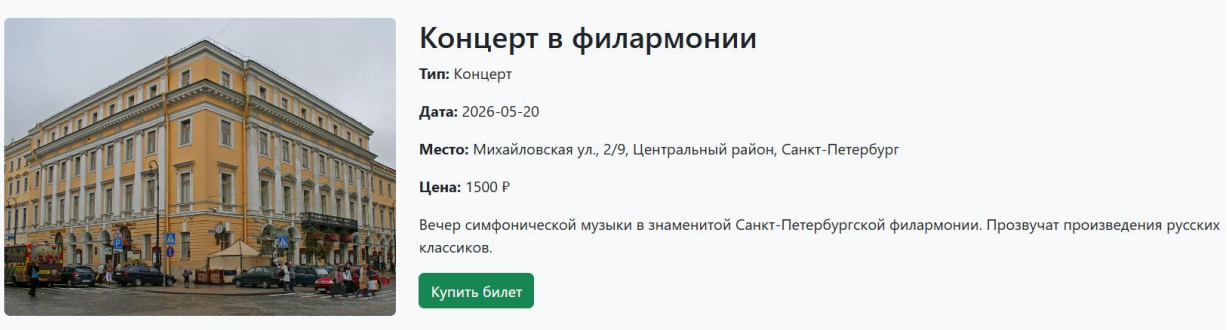


Рисунок 7 - Страница отдельного мероприятия.

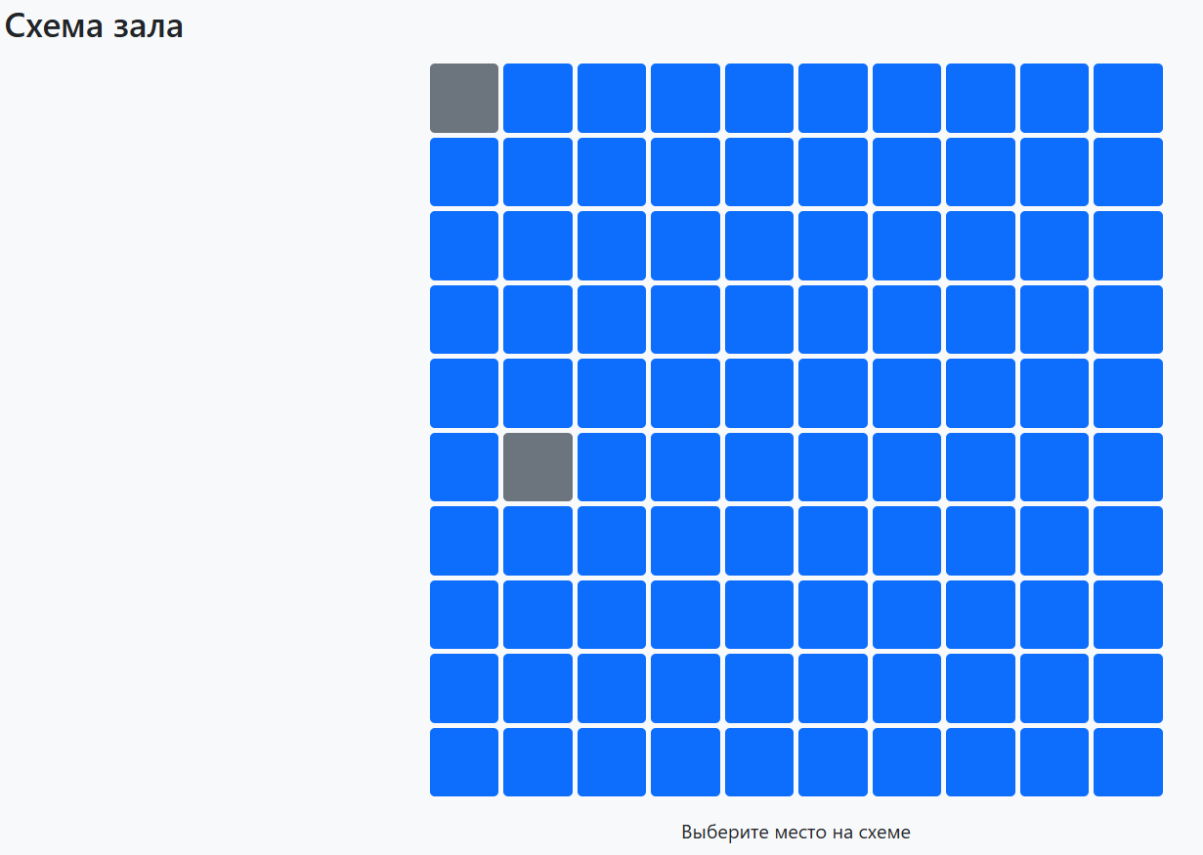


Рисунок 8 - Схема зала на странице мероприятия.

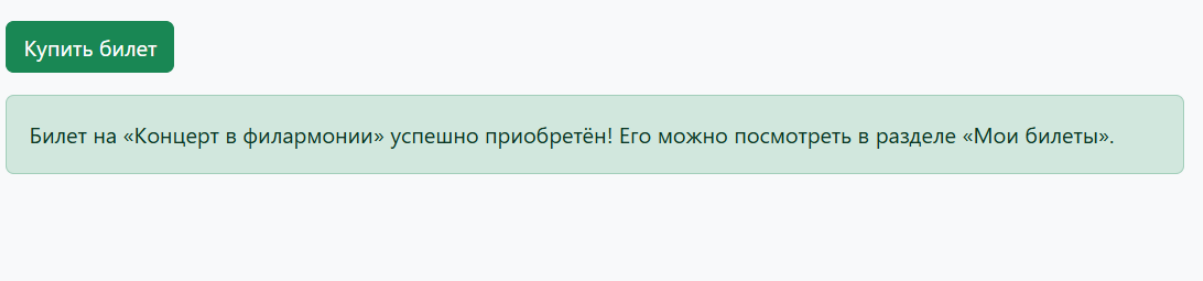


Рисунок 9 - Покупка билета через API.

7. Личный кабинет пользователя

В первой лабораторной работе личный кабинет пользователя был сделан как страница-интерфейс с блоками:

- купленные билеты;
- возвращённые билеты.

Во второй лабораторной работе этот раздел был подключён к API. Для него был создан файл dashboard-page.js.

Теперь в личном кабинете:

- загружаются купленные билеты текущего пользователя;
- отображаются возвращённые билеты;
- можно вернуть билет;
- информация подгружается после авторизации.

При возврате билета выполняются два действия:

1. билет удаляется из списка купленных;
2. информация о возврате сохраняется в коллекцию returns.

Таким образом, кабинет пользователя стал уже не макетом, а работающим разделом приложения.

По сравнению с ЛР1 здесь произошло одно из самых заметных изменений, потому что пользователь действительно взаимодействует с сервером и видит результат своих действий в интерфейсе.

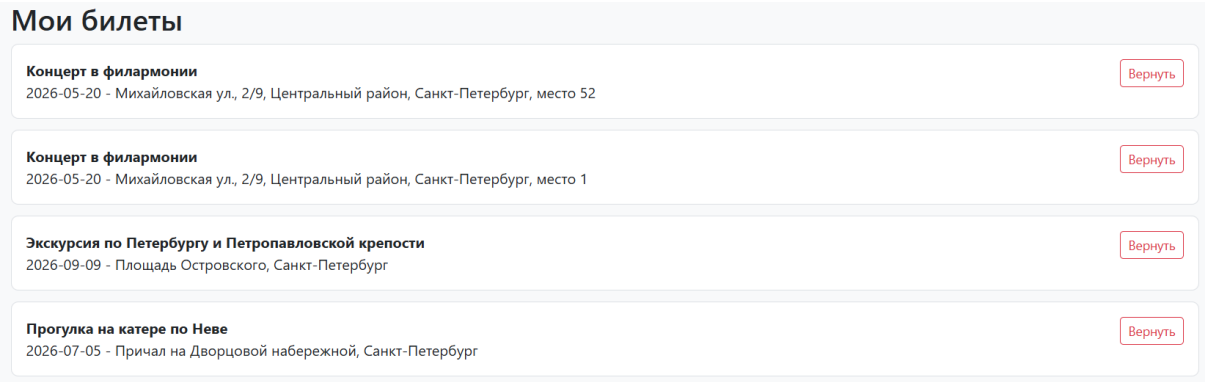


Рисунок 10 - Отображение купленных билетов.

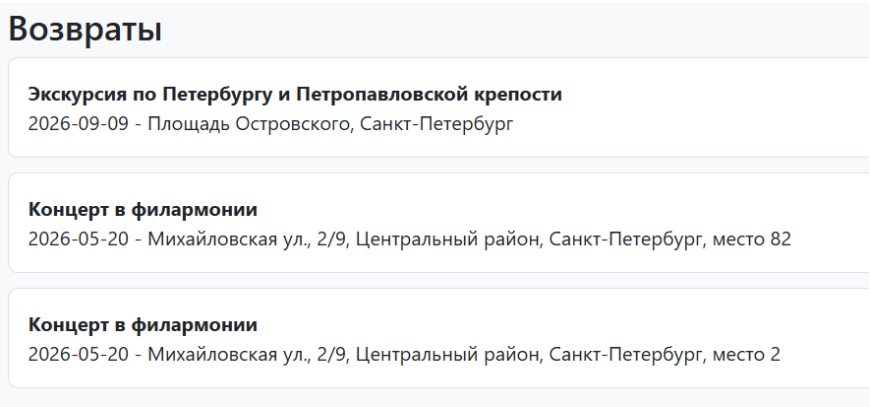


Рисунок 11 - Отображение возвращённых билетов.

8. Кабинет организатора

В первой лабораторной работе кабинет организатора тоже был сделан как интерфейс, где можно было показать список мероприятий и форму создания нового события. Во второй лабораторной работе этот раздел был подключён к API и стал полноценной рабочей частью проекта.

Для него был создан файл organizer-page.js.

На странице организатора теперь реализованы:

- загрузка мероприятий текущего организатора;

- отображение статистики по каждому мероприятию;
- количество проданных билетов;
- количество возвратов;
- форма создания нового мероприятия;
- отправка нового мероприятия на сервер.

Форма создания события содержит:

- название;
- тип;
- дату;
- место;
- цену;
- описание;
- ссылку на афишу.

После заполнения формы данные сохраняются через API и новое мероприятие появляется в общем списке.

По сравнению с ЛР1 кабинет организатора стал более реалистичным, так как теперь это не просто свёрстанный интерфейс, а часть системы с реальным добавлением данных.

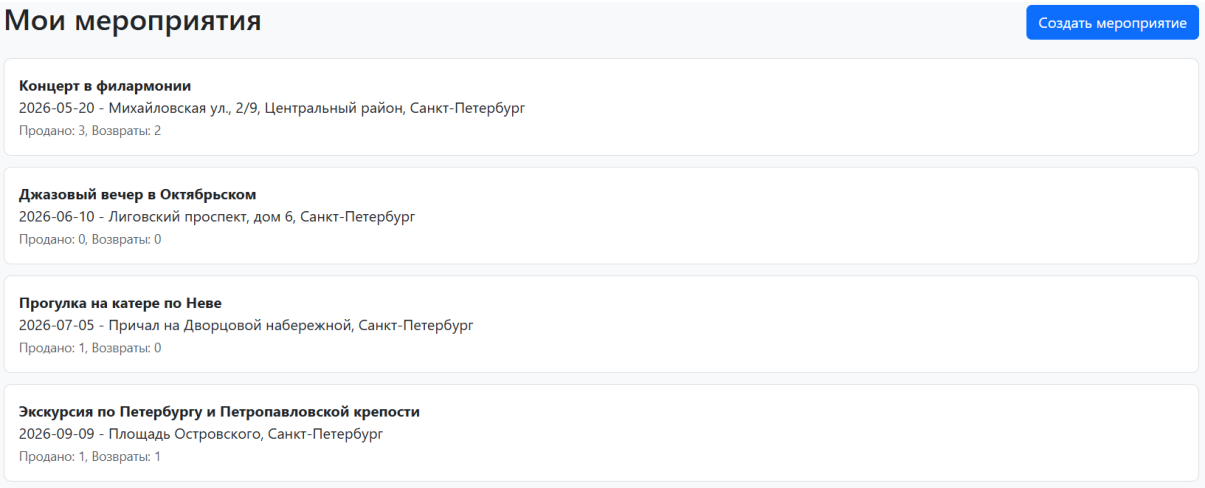


Рисунок 12 - Личный кабинет организатора.

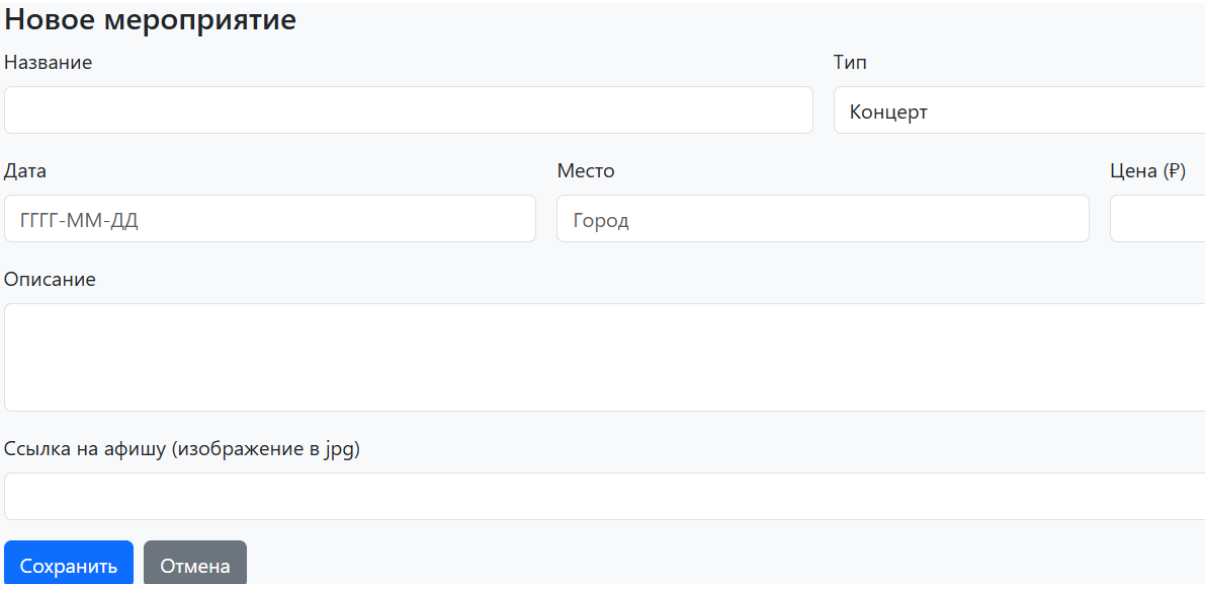


Рисунок 13 - Форма создания нового мероприятия.

9. Работа с JavaScript и взаимодействие с API

Во второй лабораторной работе основная часть доработки по сравнению с ЛР1 была связана именно с JavaScript.

Если в ЛР1 JavaScript использовался в основном для:

- обработки кнопок;

- переходов между страницами;
- фильтрации;
- базовой работы с DOM,

То во второй лабораторной работе к этому добавилось:

- выполнение GET-запросов;
- выполнение POST-запросов;
- выполнение DELETE-запросов;
- работа с защищёнными маршрутами;
- сохранение токена;
- чтение данных о текущем пользователе;
- обновление содержимого страниц после ответа сервера.

Отдельно я постаралась сделать структуру JavaScript более понятной:

- api.js отвечает за запросы к серверу;
- файлы вида *-page.js отвечают за конкретные страницы;
- auth-service.js и auth-ui.js отвечают за вход и регистрацию;
- script.js содержит общий код интерфейса.

Мне показалось, что такая структура удобнее, чем хранить весь код в одном большом файле.

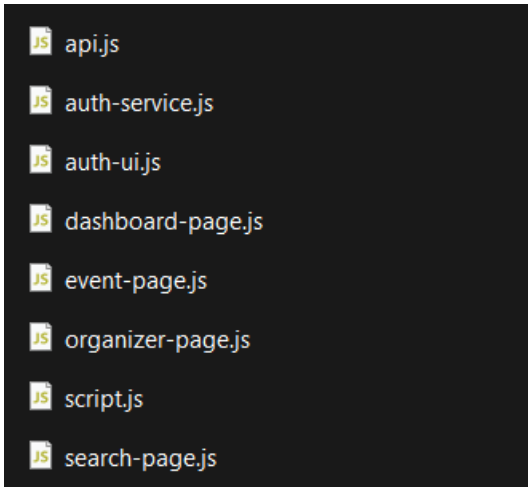


Рисунок 14 - Структура JavaScript-файлов проекта.

10. Что изменилось по сравнению с ЛР1

Во второй лабораторной работе были сохранены:

- внешний вид страниц;
- навигация;
- общая тема сайта;
- стили;
- Bootstrap-компоненты;
- общая структура интерфейса.

Но при этом были сделаны важные изменения:

1. **Подключён mock API**: вместо условной или локальной логики данные теперь подгружаются с сервера.
2. **Появилась авторизация**: теперь пользователь действительно входит в систему через API.

3. **Появилась работа с JSON-данными:** все данные хранятся в db.json.
4. **Реализована запись действий пользователя:** покупка билета, возврат, отзыв, создание события сохраняются в базе.
5. **Улучшена структура проекта:** код разбит на логические модули.
6. **Сайт стал ближе к реальному приложению:** если в ЛР1 это был в первую очередь интерфейс, то в ЛР2 это уже прототип работающего сервиса.

Вывод

В ходе выполнения лабораторной работы №2 я доработала проект по варианту **«Платформа для покупки билетов на мероприятия»**, который был начат в лабораторной работе №1, и подключила его к внешнему API.

В результате были сохранены и использованы все основные страницы, реализованные ранее:

- вход;
- регистрация;
- личный кабинет пользователя;
- поиск мероприятий;
- страница мероприятия;
- кабинет организатора.

Во второй лабораторной работе были реализованы:

- работа с форматом JSON;
- взаимодействие с API через fetch;
- mock API на основе json-server;
- авторизация через json-server-auth;
- загрузка мероприятий, билетов, возвратов и отзывов с сервера;
- создание новых записей через интерфейс;
- разграничение ролей пользователя и организатора.

По сравнению с первой лабораторной работой проект стал более функциональным и реалистичным. Если раньше он представлял собой в основном интерфейс сайта, то теперь он уже работает как небольшое приложение с серверной частью в виде mock API.

Считаю, что поставленная цель была достигнута: интерфейс из первой лабораторной работы был успешно доработан и привязан к внешнему API.